

MSPM0G3507 Competition Car Tutorial — Ball Control + 8-Channel Line Following

MSPM0G3507 Competition Car Tutorial — Ball Control + 8-Channel Line Following

- 1. Hardware Platform
- 2. Software Environment
- 3. Structure Description
 - 3.1 Component List and Material Preparation
 - 3.2 Physical Appearance
 - 3.3 Assembly Instructions
- 4. Basic Principles
 - 4.1 Wiring Pinout
 - 4.2 Physical Wiring Diagram
 - 4.3 Control Principle
- 5. Project File Structure
- 6. Core Parameter Configuration
 - 6.1 Servo Ball Control PID (app_k230.c)
 - 6.2 Line Tracking PID (app_irtracking.c)
 - 6.3 Task Line Tracking Speeds (questions.c)
 - 6.4 Task Timeout Settings (questions.c)
- 7. Task Description
 - 7.1 State Machine Structure (questions.h)
 - 7.2 Detailed Task Description
- 8. Key Code Explanation
 - 8.1 K230 Ball Control — K230_Servo_Control() (app_k230.c)
 - 8.2 Line Tracking — LineWalking() (app_irtracking.c)
 - 8.3 Line Tracking + Ball Control Combined — LineWalking_WithBall() (app_irtracking.c)
 - 8.4 Task 2 Ball Reciprocation (questions.c)
- 9. Operation Procedure

1. Hardware Platform

Component	Model/Description
Main Controller	MSPM0G3507 Development Board
Camera	K230 AI Vision Module (YOLOv8 Ball Detection)
Line Tracking	8-Channel Red Tracking Module (I2C 0x12)
Display	OLED 128×32 (I2C)
Servo	9G Servo (Ball Tilt Control)
Motors	Two L-Type 520 Encoded Motors (Serial Driver Board)
Button	Expansion Board K1 Single-Key Control

2. Software Environment

- **IDE:** Keil MDK V5 (ARM Compiler V6.7)
- **SDK:** TI MSPM0 SDK 2.02.00.05

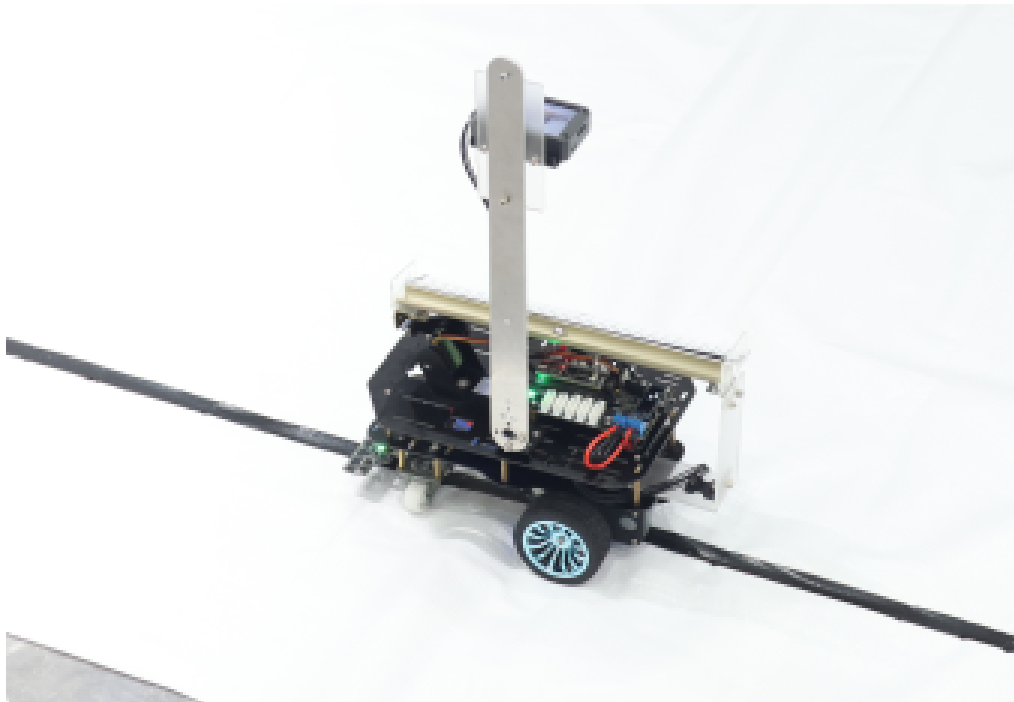
3. Structure Description

3.1 Component List and Material Preparation

- **Chassis Parts:** Black metal etched baseplate, drive motors, wheels. The chassis has reserved screw holes and wire routing holes.
- **Robotic Arm Structure:** Blue/black metal servo brackets, metal connecting rods, metal L-type/linear connectors, servos, metal flanges.
- **Horizontal Ruler Slider:** Transparent acrylic housing, handwritten digital ruler with zero reference line, internal transmission guide rails/screw rods.
- **Top Vision Bracket:** Brushed metal strips (aluminum alloy/stainless steel), transparent acrylic L-brackets, K230 module.
- **Hardware Accessories:** M2/M3 combo screws, nuts, washers, black cable ties, heat shrink tubing.

Note: In addition to the v2 chassis kit components, 9g servos, K230, and tracking modules can be purchased from our store. Other connecting rods, rulers, water pipes, and hardware accessories need to be obtained on your own.

3.2 Physical Appearance



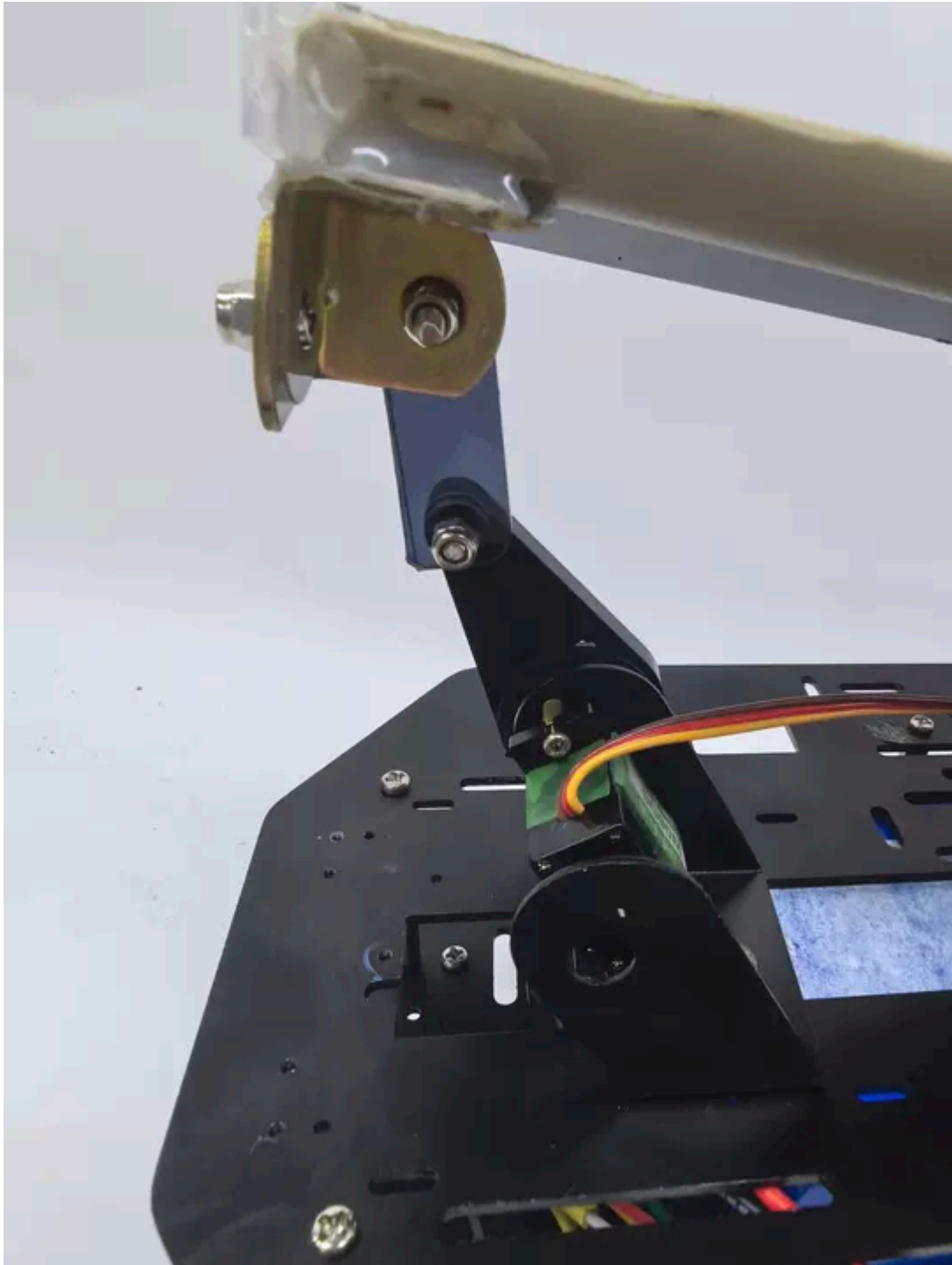












3.3 Assembly Instructions

Step 1: Install Mobile Chassis and Motors

- Install the two drive motors (with wheels) on both sides of the black metal baseplate. Ensure the wheels face the same direction and make full contact with the ground.
- Install the main controller board MSPM0G3507 and secure it with screws at the reserved position on the chassis. Note that the main controller board should avoid the wheel and motor rotation space.

Step 2: Build Horizontal Ruler and Translation Stage

- *Reference Figure 5:* Assemble the transparent acrylic housing and insert the cardboard with the graduated ruler inside. This structure is fixed at the front of the chassis.

- Note that the guide rails or screw rods inside the slider need to remain horizontal, which is the key reference for subsequent robotic arm coordinate calibration.

Step 3: Assemble Robotic Arm Joints

- *Reference Figures 3 and 7:* Insert the green servo into the black metal bracket and secure it with matching screws.
- Install the servo arm and connect the blue/black metal connecting rods.
- Build the end effector (connector/claw) as shown in Figure 7. In the image, this component uses a lot of **transparent acrylic and hot glue/resin** for reinforcement. In actual assembly, it is recommended to use 3D printed parts or metal parts instead to ensure sufficient structural strength.
- Secure the servo base to the rear of the chassis (refer to the layout position in Figure 7).

Step 4: Install Vertical Metal Bracket

- Reference Figures 1, 4, and 6: Take the brushed metal strip and install it vertically into the fixed slot at the center or side-front of the chassis.
- When securing with screws, ensure the metal strip is perpendicular to the chassis plane and does not wobble. The nuts in Figure 6 are locked through from the back.

Step 5: Install Top Vision Sensor

- Reference Figure 35: Face the K230 module's back side outward. Use transparent acrylic L-brackets and M3 screws to secure the module at the top of the vertical metal bracket.
- Note to adjust the sensor orientation to ensure it can see the track and ruler ahead.

4. Basic Principles

4.1 Wiring Pinout

4-Channel Motor Driver Board:

Motor Driver	MSPM0G3507
RX2	PB6
TX2	PB7
GND	GND
5V	5V

Motor Wiring:

Motor	Driver Board
M2	M-
V	3V3
A	H1B

Motor	Driver Board
B	H1A
G	GND
M1	M+

8-Channel Line Tracking Module:

Tracking Module	MSPM0G3507
5V	5V
SCL	PA15
SDA	PA16
GND	GND

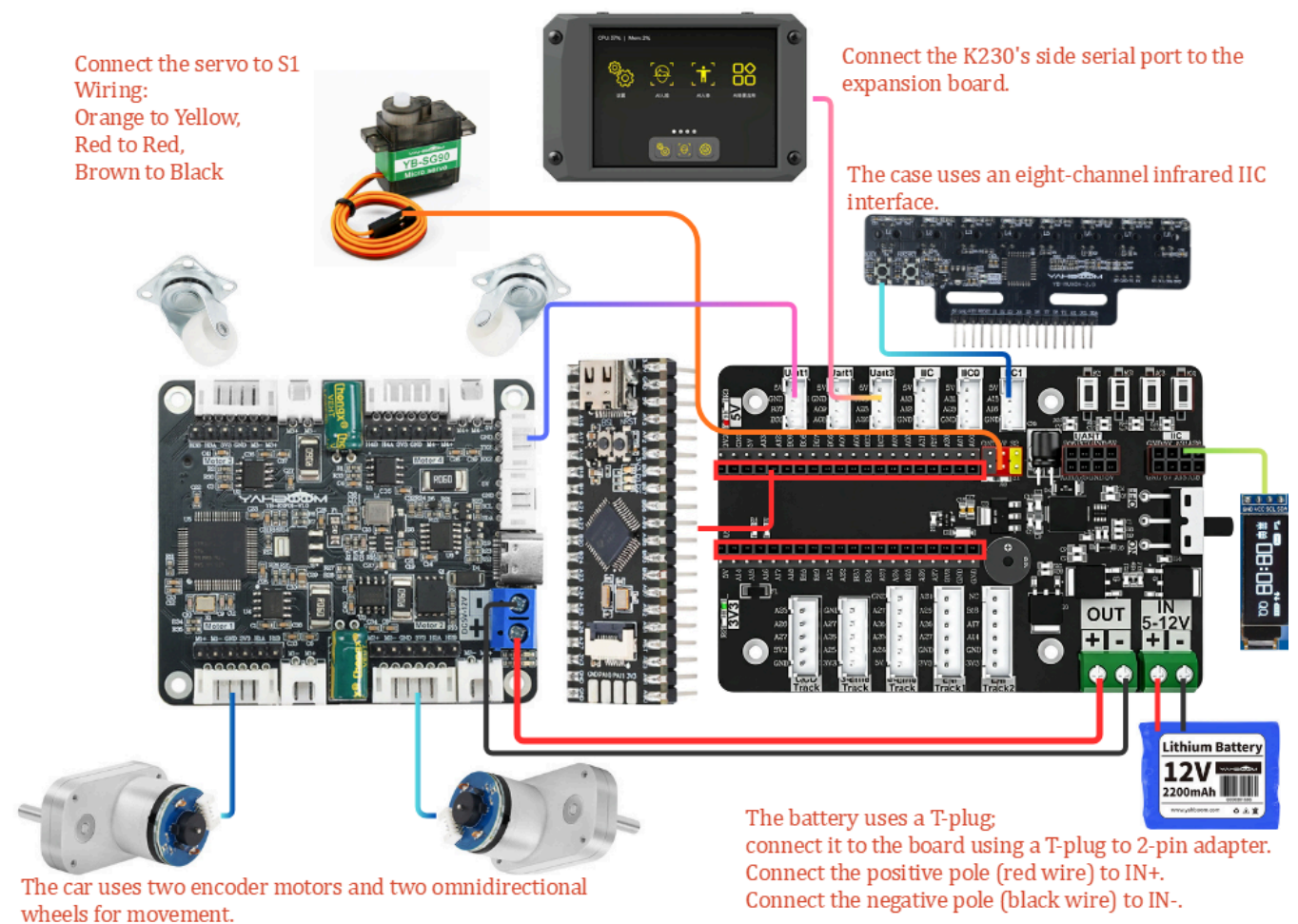
OLED:

OLED	MSPM0G3507
VCC	5V
GND	GND
SDA	PA28
SCL	PA31

K230 Camera:

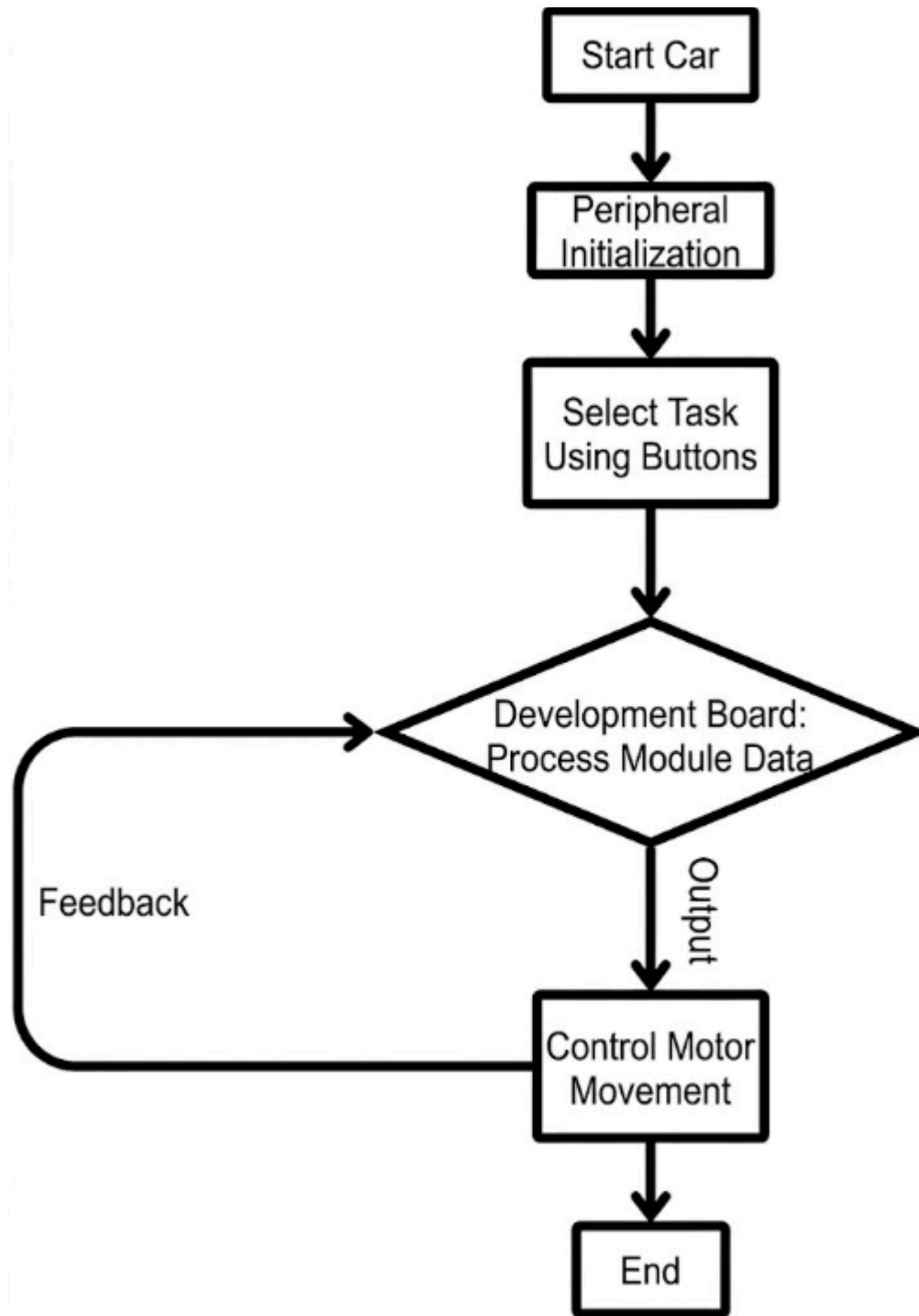
K230	MSPM0G3507
5V	5V
GND	GND
TXD	PA3
RXD	PA2

4.2 Physical Wiring Diagram



4.3 Control Principle

Car Line Following Control Flow:



1. 8-channel infrared sensor reads black line position -> Calculate deviation value
2. PID controller calculates turn correction -> Control motor differential steering
3. K230 camera detects ball position -> Servo PID controls rod angle
4. Timer timeout or waypoint detection -> Stop/Clear state transition

PID Control Parameter Description:

PID Parameter	Function	Adjustment Direction
P (Proportional)	Determines response speed, larger output when deviation is large	Too large: Oscillating snake pattern; Too small: Weak turns

PID Parameter	Function	Adjustment Direction
I (Integral)	Eliminates steady-state deviation, cumulative error correction	Too large: Overshoot; Too small: Drifts to one side
D (Derivative)	Suppresses oscillation, early braking	Too large: Jitter; Too small: Can't stop in time

5. Project File Structure

```
Competition_PJ/
├─ main.c                # Main program: Peripheral initialization, main loop, state
machine dispatch
├─ APP/
│   ├─ app_k230.c/h      # K230 serial protocol + PID servo ball control (core)
│   ├─ app_pid.c/h       # PID controller (positional + incremental)
│   ├─ app_kalman.c/h    # 1D Kalman filter
│   └─ app_servo.c/h     # Servo PWM drive
├─ BSP/
│   ├─ Eight_Tracking/
│   │   └─ app_irtracking.c/h # 8-channel line tracking + tracking PID + combined tasks
(tracking + ball control)
│   ├─ Motor/
│   │   ├─ app_motor.c/h   # Car motion control (Motion_Car_Control)
│   │   └─ app_motor_uart.c/h # Motor driver serial protocol
│   ├─ OLED/
│   │   ├─ oled.c/h       # OLED driver + display function OLED_time()
│   │   └─ oledfont.h     # Font data
│   ├─ Key/
│   │   └─ key.c/h        # Button driver + mode selection state machine
│   ├─ Questions/
│   │   └─ questions.c/h  # Competition task state machine (Task 1~5)
│   ├─ Task/
│   │   └─ task.c/h       # Timed task scheduler (Scheduler_Run)
│   ├─ Timer/
│   │   └─ timer.c/h      # 1ms system clock Get_Time()
│   └─ uart.c/h          # Serial initialization (UART0 debug, UART1 motor, UART3
K230)
└─ ti_msp_dl_config.c/h  # TI driver library configuration (SysConfig generated)
```

6. Core Parameter Configuration

6.1 Servo Ball Control PID (app_k230.c)

```
#define PID_KP      0.08f    // Proportional coefficient: Response force, larger =
stronger push
#define PID_KI      0.003f   // Integral coefficient: Eliminate steady-state offset
#define PID_KD      2.86f    // Derivative coefficient: Main damping, suppress oscillation
```

```

#define K230_CENTER    360.0f  // K230 image center X pixel
#define K230_MIN       99.0f   // Left boundary pixel
#define K230_MAX       599.0f  // Right boundary pixel
#define FOV_CM         28.0f   // Camera field of view width (cm)
#define PIPE_HALF_CM   12.5f   // Pipe half-length (cm), full pipe length 25cm

#define DEADBAND_PX    3.0f    // Deadband (px), no action when error < 3px
#define RATE_LIMIT     6.0f    // Single frame CC change limit, prevent servo jerking
#define KALMAN_Q        1.0f   // Kalman process noise (larger = faster response)
#define KALMAN_R        0.3f   // Kalman measurement noise (larger = smoother)

```

Bidirectional px_per_cm Compensation (camera not perfectly centered):

```

Target >= 0 (right side): px_per_cm = (599-360)/12.5 = 19.12
Target < 0 (left side):  px_per_cm = (360-99)/12.5  = 20.88

```

PID Tuning Guide:

Phenomenon	Adjust	Direction	Description
Ball oscillates near center	↑ KD	Increase damping	Larger KD brakes earlier
Ball stops before reaching target	↑ KP	Increase push	Larger KP pushes farther
Ball overshoots and bounces far	↓ KP	Decrease push	
Ball stable but biased to one side	↑ KI	Strengthen integral	Larger integral returns to center faster
Servo jitters when ball is still	↑ DEADBAND	Loosen deadband	

6.2 Line Tracking PID (app_irtracking.c)

```

#define IRTrack_Trun_KP (150)  // Line tracking P: Larger = stronger correction
#define IRTrack_Trun_KI (0.15) // Line tracking I: Eliminate steady-state deviation
#define IRTrack_Trun_KD (1.5)  // Line tracking D: Suppress oscillation
#define IRR_SPEED      250     // Line tracking default speed (mm/s)

```

Line Tracking Tuning Guide:

Phenomenon	Adjust	Direction
Insufficient turn / Pressing line	↑ KP	Increase
Snake-like movement	↑ KD or ↓ KP	Damping / Reduce push

Phenomenon	Adjust	Direction
Constant drift to one side	↑ KI	Strengthen integral

Inertia Compensation (for combined tracking + ball control task):

```
#define BALL_INERTIA_FWD    -0.8f    // Forward inertia compensation (cm), larger value pushes
ball more forward
#define BALL_INERTIA_TURN  -0.8f    // Turn inertia compensation (cm)
#define BALL_START_COMP    -5.0f    // Start compensation (cm), push forward when car
accelerates and ball lags
#define BALL_STOP_COMP      5.0f    // Stop compensation (cm), pull back when car stops and
ball overshoots
#define RAMP_TIME_MS       3000    // Start ramp-up time (ms), larger = gentler start
```

6.3 Task Line Tracking Speeds (questions.c)

```
#define TASK1_SPEED        316    // Task 1 pure tracking speed
#define TASK3_SPEED        130    // Task 3 A->B + ball control speed
#define TASK4_SPEED        222    // Task 4 one lap + ball control speed
#define TASK5_SPEED        222    // Task 5 specified position speed
#define RAMP_TIME_MS       3000    // Start ramp-up time
```

6.4 Task Timeout Settings (questions.c)

```
#define TASK1_MAX_MS       20000   // Task 1 pure tracking 20s timeout
#define TASK2_MAX_MS       5000    // Task 2 ball reciprocation 5s timeout
#define TASK3_MAX_MS       8000    // Task 3 A->B + ball control 8s timeout
#define TASK4_MAX_MS       30000   // Task 4 one lap + ball control 30s timeout
#define TASK5_MAX_MS       30000   // Task 5 specified position 30s timeout
```

7. Task Description

7.1 State Machine Structure (questions.h)

Main State (Main_State):

Macro	Value	Task Description
STOP_STATE	0	Stop
TASK_LINE_LAP	1	Task 1: Pure tracking one lap return to A, ≤20s
TASK_BALL_CTRL	2	Task 2: Ball O→+5→0→-5 (stop), <5s
TASK_AB_STAB	3	Task 3: A→B + ball stable at O, ≤8s

Macro	Value	Task Description
TASK_LAP_STAB	4	Task 4: One lap return to A + ball stable at O, ≤30s
TASK_LAP_POS	5	Task 5: One lap return to A + ball at specified position, ≤30s

Sub State (Sub_State):

Macro	Value	Description
SUB_IDLE	0	Idle
SUB_INIT	1	Initialize (set speed, start timer)
SUB_RUNNING	2	Running
SUB_DONE	3	Done (stop car but maintain ball control, timeout also enters this state)

7.2 Detailed Task Description

Task 1 (Pure Line Tracking):

- Line tracking only, no ball control
- Speed: TASK1_SPEED = 316 mm/s
- Pass through 4 waypoints (A/B/C/D) and stop at point A
- Auto-stop after 20s timeout

Task 2 (Ball Control):

- Car stays still, servo control only
- Flow: O → +5cm (stable) → 0cm (stable) → -5cm (stable)
- Each phase waits for ball to approach target (≤0.5cm) for 30 consecutive frames before transitioning
- 5s timeout
- After task completion, maintain ball at -5cm position

Task 3 (A→B + Ball Control):

- Line tracking + servo ball control simultaneously
- Speed: TASK3_SPEED = 130 mm/s
- Start from point A, pass through 1 waypoint to reach point B
- Ball remains stable at center O during travel
- Stop after 8s timeout, maintain ball control after stopping

Task 4 (One Lap + Ball Control):

- Line tracking + servo ball control simultaneously
- Speed: TASK4_SPEED = 222 mm/s
- Clockwise one lap, pass through 4 waypoints and return to A
- Ball remains stable at center O during travel

- Stop after 30s timeout, maintain ball control after stopping

Task 5 (Specified Position + Ball Control):

- Same as Task 4, but ball target position is configurable (Target_Position)
- Speed: TASK5_SPEED = 222 mm/s
- Stop after 30s timeout, maintain ball control after stopping

8. Key Code Explanation

8.1 K230 Ball Control — K230_Servo_Control() (app_k230.c)

```
void K230_Servo_Control(float target_cm)
{
    // Parse k230 serial frame (UART3 interrupt auto-receives data)
    int r; do { r = parse_one(g_buf, g_wr); } while (r > 0);
    if (!g_valid) return;

    // Coordinate conversion cm -> pixels (bidirectional px/cm compensation)
    float px_per_cm;
    if (target_cm >= 0.0f)
        px_per_cm = (K230_MAX - K230_CENTER) / PIPE_HALF_CM; // Right side
    else
        px_per_cm = (K230_CENTER - K230_MIN) / PIPE_HALF_CM; // Left side
    float target_px = K230_CENTER + target_cm * px_per_cm;

    // Deadband + positional PID + rate limiting + PWM output
    float error = target_px - kalman_Filter(&kf_x, g_x);
    if (error > -DEADBAND_PX && error < DEADBAND_PX) return;
    float cc = PID_To_CC_Pos(&g_pid, x_flt);
    // ... rate limiting + CC limits ...
    DL_TimerA_setCaptureCompareValue(PWM_Servo_INST, (uint16_t)cc, DL_TIMER_CC_0_INDEX);
}
```

PID Formula (Positional, app_pid.c): $u = K_p * e + K_i * \text{Integral}(e) + K_d * (e - e_{\text{last}})$

8.2 Line Tracking — LineWalking() (app_irtracking.c)

```
void Linewalking(void)
{
    static int8_t err = 0;
    static u8 x1~x8;
    deal_IRdata(&x1, ..., &x8); // I2C read 8-channel infrared

    if (x4 == 0 || x5 == 0) err = 0; // Center black line, go straight
    else if (x3 == 0) err = -1; // Deviate left, turn right
    else if (x6 == 0) err = 1; // Deviate right, turn left
    // ... more sensor combinations ...

    pid_output_IRR = (int)(PID_IR_Calc(err));
}
```

```

    Motion_Car_Control(g_line_speed, 0, pid_output_IRR);
}

```

8.3 Line Tracking + Ball Control Combined — Linewalking_WithBall() (app_irtracking.c)

```

void Linewalking_WithBall(float ball_target_cm)
{
    // Inertia compensation: Forward + turning
    float compensated = ball_target_cm;
    compensated -= BALL_INERTIA_FWD;
    if (pid_output_IRR > 0)
        compensated -= BALL_INERTIA_TURN * pid_output_IRR / 5000.0f;

    // Start ramp-up: Over first RAMP_TIME_MS milliseconds, speed gradually increases from 0
    uint32_t et = Get_Time() - start_time;
    if (et < RAMP_TIME_MS) {
        float ramp = (float)et / RAMP_TIME_MS;
        g_line_speed = (int)(g_line_speed * ramp);
    }

    Linewalking();                // Line tracking
    K230_Receive_Process();       // Receive K230 data
    K230_Servo_Control(compensated); // Ball control
}

```

8.4 Task 2 Ball Reciprocation (questions.c)

```

// Three phases: +5cm(1.8s) -> 0cm(1.8s) -> -5cm(1.8s) -> DONE(maintain -5)
case SUB_RUNNING: {
    static int phase = 0;
    static uint32_t t0 = 0;
    if (t0 == 0) t0 = Get_Time();

    float target_cm;
    if (phase == 0) target_cm = +5.0f;
    else if (phase == 1) target_cm = 0.0f;
    else target_cm = -5.0f;

    K230_Receive_Process();
    K230_Servo_Control(target_cm);

    if (Get_Time() - t0 >= 1800) { // 1.8 seconds per phase
        t0 = Get_Time();
        if (phase == 0) phase = 1;
        else if (phase == 1) phase = 2;
        else { phase = 0; Sub_State = SUB_DONE; }
    }
    break;
}

case SUB_DONE:

```

```
K230_Receive_Process();  
K230_Servo_Control(-5.0f);    // Maintain ball at -5cm position  
break;
```

9. Operation Procedure

1. After wiring, power on. OLED displays "K230 Init..." and waits 4 seconds
2. Enter mode selection interface, OLED displays "Mode:1"
3. **Long press K1** to switch modes (1~5)
4. **Short press K1** to confirm and enter current mode
5. Car starts executing task, OLED displays task name + timer
6. **Short press K1 during task:** Pause; **Long press K1:** Exit and return to mode selection
7. After task completion/timeout, car stops but **ball control task continues** (ball stays at target position)
8. Long press K1 to exit and return to mode selection